Práctica 4: Coach Financiero Generativo con Agentes LangGraph

Autores:

Diego Esclarín (51729611N)
Sofía Contreras (09848471D)

Aplicaciones de Inteligencia Artificial
Grado en Ingeniería en Inteligencia Artificial
Universidad Rey Juan Carlos
Curso 2025–26

Índice

1. Introducción **7**
1.1. Objetivos del Proyecto . . . 7
1.2. Alcance y Limitaciones . . . 7

2. Contexto y Motivación **9**
2.1. Problemática Actual . . . 9
2.2. Fundamentos Tecnológicos . . . 9
2.3. Motivación del Proyecto . . . 9

3. Estado del Arte **11**
3.1. Sistemas Tradicionales de Gestión Financiera . . . 11
3.2. Técnicas de Procesamiento de Lenguaje Natural en Finanzas . . . 11
3.3. Agentes Conversacionales con Herramientas . . . 11
3.4. Arquitecturas de Agentes Basados en Grafos . . . 11
3.5. Posicionamiento de la Propuesta . . . 12

4. Propuesta y Desarrollo **13**
4.1. Arquitectura General del Sistema . . . 13
4.2. El Agente LangGraph . . . 13
4.2.1. Grafo de estado . . . 13
4.2.2. Estado del grafo . . . 14
4.2.3. Descripción de los nodos . . . 14
4.3. Las Herramientas del Agente . . . 15
4.4. Sistema de Memoria Persistente . . . 16
4.4.1. Estructura de datos . . . 16
4.4.2. Sistema de alertas . . . 17
4.5. Nodo de Resumen Automático . . . 17
4.5.1. Motivación . . . 17
4.5.2. Funcionamiento . . . 17
4.5.3. Parámetros configurables . . . 17
4.6. El Coach Clásico como Línea Base . . . 18
4.7. Comparativa: Coach Clásico vs. Agente LangGraph . . . 18
4.8. Interfaz de Usuario . . . 19
4.9. Implementación Técnica . . . 19

5. Conclusiones y Trabajo Futuro **21**
5.1. Conclusiones Técnicas . . . 21
5.2. Contribuciones del Proyecto . . . 21
5.3. Implicaciones Sociales . . . 21
5.4. Trabajo Futuro . . . 22

Índice de figuras

1. Arquitectura de capas del sistema coach financiero. 13
2. Grafo LangGraph del agente financiero. El bucle ReAct (agent/tools) se repite hasta que el LLM no emite llamada a herramienta. 14
3. Ciclo de vida del resumen automatico a lo largo de sesiones sucesivas. El nodo `summarize` se activa en la Sesion 3 cuando el umbral de 20 mensajes se supera. 18

Índice de cuadros

1. Nodos del grafo LangGraph y su función. 15
2. Las trece herramientas del agente financiero. 16
3. Comparativa entre el Coach Clásico y el Agente LangGraph. 19

1. Introducción

El propósito fundamental de este proyecto es el desarrollo de un sistema de asesoramiento financiero personal basado en agentes de inteligencia artificial generativa. En un entorno donde el volumen de transacciones digitales crece exponencialmente [1], la capacidad de analizar datos financieros de manera inteligente deja de ser un lujo para convertirse en una necesidad técnica. El sistema diseñado actúa como un coach financiero activo capaz de comprender consultas en lenguaje natural, analizar patrones de gasto y proporcionar recomendaciones personalizadas.

El núcleo de la investigación se centra en la implementación de un agente inteligente utilizando la arquitectura LangGraph [2], que permite al sistema tomar decisiones dinámicas sobre qué información consultar y cómo responder a las consultas del usuario. A diferencia de los sistemas basados en reglas estáticas [3], el agente desarrollado posee la capacidad de razonar sobre los datos financieros mediante un bucle de razonamiento-acción (ReAct [4]) y mantener un estado persistente entre sesiones.

Finalmente, el proyecto destaca por su capacidad de adaptación continua mediante mecanismos de memoria persistente [5]. Esta característica permite al sistema evolucionar con el usuario, recordando objetivos financieros y generando alertas contextualizadas, lo que lo posiciona como una solución innovadora en el campo de la tecnología financiera personal [6].

1.1. Objetivos del Proyecto

Los objetivos principales de esta práctica se estructuran en tres niveles complementarios:

Objetivo técnico: Diseñar e implementar un agente conversacional basado en LangGraph capaz de analizar datos financieros personales utilizando un conjunto de trece herramientas especializadas. El sistema debe demostrar capacidad de razonamiento multi-paso, encadenamiento de herramientas, gestión de la ventana de contexto mediante resumen automático y mantenimiento de estado persistente entre sesiones.

Objetivo funcional: Desarrollar una aplicación funcional que permita a los usuarios establecer objetivos de ahorro por categoría, recibir alertas automáticas cuando se acercan al límite, consultar análisis detallados de sus gastos y obtener recomendaciones financieras contextualizadas basadas en su historial de transacciones real.

Objetivo comparativo: Mantener una implementación clásica basada en clasificación de intenciones por keywords como línea base, permitiendo evaluar cuantitativamente y cualitativamente las mejoras introducidas por la arquitectura de agente frente al enfoque determinista tradicional.

1.2. Alcance y Limitaciones

El proyecto se centra exclusivamente en el análisis de datos financieros personales estructurados, utilizando como fuente principal un conjunto de transacciones bancarias en formato CSV heredado de la Práctica 2 de la asignatura (aproximadamente 900 transacciones reales).

Las limitaciones principales del sistema incluyen: la dependencia de la calidad y completitud de los datos de entrada, la ausencia de integración con APIs bancarias en tiempo real, y la dependencia del servicio externo Groq API para la inferencia del modelo de lenguaje.

2. Contexto y Motivación

La gestión financiera personal representa uno de los desafíos más significativos de hoy en día. El volumen de transacciones digitales crece exponencialmente [1], mientras que los usuarios se enfrentan diariamente a la necesidad de comprender sus patrones de gasto y establecer objetivos de ahorro realistas. Sin embargo, las herramientas disponibles ofrecen o bien análisis retrospectivo básico o requieren conocimientos técnicos avanzados para su configuración y uso efectivo.

2.1. Problemática Actual

El panorama actual de las aplicaciones de gestión financiera personal presenta limitaciones significativas en tres dimensiones fundamentales.

Limitaciones de interpretación: Las soluciones comerciales ofrecen dashboards genéricos que presentan datos de forma visualmente atractiva pero carecen de capacidad de interpretación contextual [3]. Estas herramientas pueden mostrar que un usuario ha gastado un 20 % más en restaurantes, pero son incapaces de explicar por qué ha ocurrido este aumento, qué tendencia histórica lo respalda o cómo podría optimizarse.

Barreras de usabilidad: Las soluciones basadas en hojas de cálculo y software de contabilidad requieren un alto grado de conocimiento técnico. Esta barrera limita la adopción de prácticas financieras saludables y perpetúa la toma de decisiones basada en intuición en lugar de análisis de datos objetivos [6].

Ausencia de continuidad: Los sistemas existentes carecen de memoria entre sesiones. Cada interacción parte de cero, lo que impide que el sistema aprenda las preferencias del usuario, recuerde objetivos pactados o detecte cambios significativos en los patrones de gasto a lo largo del tiempo.

2.2. Fundamentos Tecnológicos

La reciente evolución de los modelos de lenguaje grandes (LLMs) y las arquitecturas de agentes basados en grafos ha abierto nuevas posibilidades en el campo de la asistencia financiera personal [7]. Estos sistemas combinan la capacidad de comprensión del lenguaje natural con el razonamiento estructurado, permitiendo crear interfaces conversacionales que pueden entender consultas complejas y realizar análisis multi-paso.

La arquitectura LangGraph [2] ofrece un marco ideal para este tipo de aplicaciones al permitir el diseño de agentes que deciden dinámicamente qué herramientas utilizar, encadenan múltiples operaciones y mantienen estado entre interacciones. A diferencia de los sistemas basados en pipelines fijos, los agentes LangGraph adaptan su comportamiento a las necesidades específicas de cada consulta mediante un grafo de nodos y aristas que define el flujo de control.

2.3. Motivación del Proyecto

Este proyecto surge de la confluencia de dos motivaciones complementarias. Por un lado, la necesidad técnica de explorar las capacidades de los agentes con herramientas en un dominio real y bien acotado. Por otro, el objetivo práctico de crear una herramienta útil para el usuario final que supere las limitaciones de los enfoques actuales.

3. Estado del Arte

El campo de la tecnología financiera personal (FinTech) ha evolucionado significativamente en la última década [1]. Esta sección analiza las principales aproximaciones tecnológicas existentes y posiciona nuestra propuesta dentro del panorama actual.

3.1. Sistemas Tradicionales de Gestión Financiera

Las aplicaciones convencionales se clasifican en tres categorías principales. Las hojas de cálculo y software de contabilidad ofrecen flexibilidad pero requieren intervención manual constante [6]. Las aplicaciones de agregación bancaria como Mint o YNAB automatizan la importación de datos pero mantienen un enfoque reactivo, limitándose a presentar información histórica sin capacidad de interpretación contextual [3]. Los robo-advisors automatizan inversiones pero su ámbito se restringe exclusivamente a la gestión de patrimonios, sin capacidad para asesoramiento sobre gastos cotidianos.

3.2. Técnicas de Procesamiento de Lenguaje Natural en Finanzas

La aplicación de PLN al dominio financiero ha abierto nuevas posibilidades [8]. Los primeros sistemas utilizaron técnicas de clasificación supervisada como Naive Bayes y SVM, logrando precisiones aceptables (70–85 %) pero con rigidez ante nuevos patrones. El análisis de sentimiento ha permitido detectar estrés financiero, aunque enfrenta desafíos por la subjetividad del lenguaje. Las técnicas de NER con modelos basados en transformers como BERT han mejorado la precisión en extracción de entidades, pero requieren grandes volúmenes de datos etiquetados.

El enfoque de clasificación de intenciones por keywords, utilizado en nuestra línea base, representa una solución ligera y determinista adecuada para dominios bien acotados. Sus limitaciones aparecen ante preguntas compuestas o con vocabulario no anticipado.

3.3. Agentes Conversacionales con Herramientas

Los recientes avances en LLMs han impulsado el desarrollo de agentes conversacionales que no solo procesan lenguaje sino que también ejecutan acciones [7]. El paradigma ReAct (Reasoning and Acting) [4], publicado en 2022, establece el marco teórico fundamental: el agente alterna entre pasos de razonamiento (*Thought*) y pasos de acción (*Action*), observando los resultados antes de continuar. Este bucle permite al modelo descomponer problemas complejos en subproblemas manejables.

La generación más reciente de agentes con herramientas representa un avance fundamental [9]. Sistemas como AutoGPT demostraron las posibilidades pero también las limitaciones de los agentes completamente autónomos. El framework LangChain introdujo una abstracción de herramientas más controlada, donde el desarrollador define las herramientas disponibles y el LLM decide cuándo y cómo usarlas.

3.4. Arquitecturas de Agentes Basados en Grafos

LangGraph emerge como una de las arquitecturas más prometedoras para el desarrollo de agentes complejos [2]. A diferencia de los sistemas sin estado, LangGraph

permite mantener información contextual entre múltiples interacciones, fundamental para asistentes financieros personales [5]. La arquitectura de grafos permite encadenar múltiples operaciones de razonamiento y seleccionar dinámicamente qué herramientas utilizar basándose en el contexto específico de cada consulta.

La gestión de la ventana de contexto es un desafío crítico en sistemas con conversaciones largas. Las estrategias existentes incluyen el recorte por ventana deslizante [5], el resumen automático mediante LLM y el filtrado de mensajes de herramienta intermedios. Nuestra implementación combina el resumen automático con persistencia entre sesiones, permitiendo que el sistema mantenga continuidad temporal indefinida sin degradar la calidad de respuesta.

3.5. Posicionamiento de la Propuesta

Frente a las soluciones existentes, nuestra propuesta se distingue por tres características que no aparecen juntas en los papers revisados: (1) el uso de un grafo de agente especializado en finanzas personales con herramientas de dominio específico, (2) la memoria persistente con sistema de objetivos y alertas automáticas, y (3) la comparativa directa entre el enfoque clásico por intenciones y el agente moderno, facilitando la evaluación rigurosa de la mejora introducida.

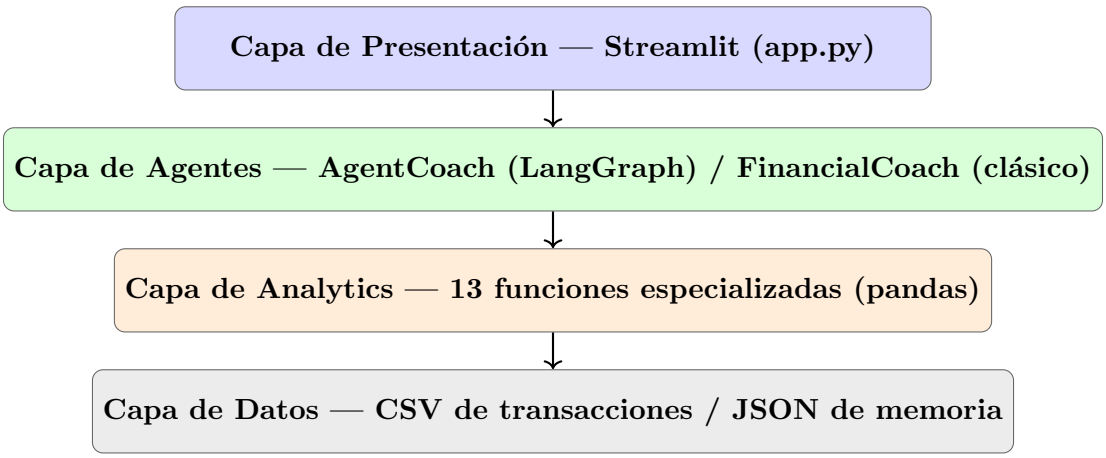

Figura 1: Arquitectura de capas del sistema coach financiero.

4. Propuesta y Desarrollo

La propuesta técnica se fundamenta en el diseño de un agente conversacional inteligente especializado en asesoramiento financiero personal. El sistema combina procesamiento de lenguaje natural con análisis financiero estructurado, implementando una arquitectura dual que permite comparar un agente LangGraph frente a una implementación clásica basada en clasificación de intenciones.

4.1. Arquitectura General del Sistema

El sistema sigue una arquitectura modular con cuatro capas principales, como se ilustra en la Figura 1.

La capa de presentación proporciona una interfaz web interactiva con métricas financieras en tiempo real, gráficos de distribución de gasto, panel de objetivos activos y un toggle que permite cambiar entre el modo agente y el modo clásico sin reiniciar la sesión.

La capa de agentes contiene dos implementaciones paralelas cuyos detalles se describen en las subsecciones siguientes. La capa de analytics implementa el motor de análisis financiero con trece funciones especializadas reutilizables por ambas implementaciones. La capa de datos gestiona la carga y procesamiento de transacciones mediante pandas, y la persistencia de memoria de usuario en archivos JSON individuales.

4.2. El Agente LangGraph

4.2.1. Grafo de estado

El componente central implementa un grafo dirigido de seis nodos conectados mediante aristas fijas y condicionales. La Figura 2 muestra su estructura completa.

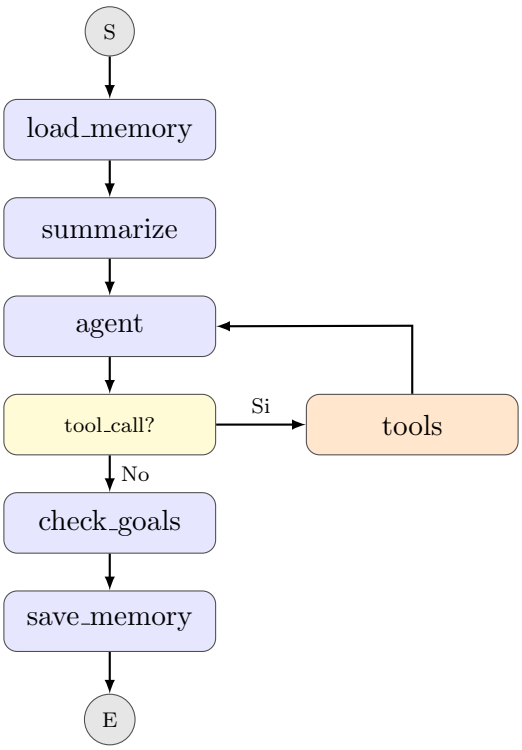

Figura 2: Grafo LangGraph del agente financiero. El bucle ReAct (agent/tools) se repite hasta que el LLM no emite llamada a herramienta.

4.2.2. Estado del grafo

El estado compartido entre nodos se define mediante un `TypedDict` que agrupa tres campos: la lista de mensajes de conversación (con un reducer acumulativo `add_messages`), la memoria persistente del usuario (objetivos, alertas emitidas y resumen acumulado) y las alertas generadas en la ejecución actual. El reducer `add_messages` de LangGraph asigna identificadores únicos a cada mensaje y gestiona su eliminación mediante `RemoveMessage`, lo que es fundamental para el nodo de resumen.

4.2.3. Descripción de los nodos

La Tabla 1 describe la responsabilidad de cada nodo del grafo.

Cuadro 1: Nodos del grafo LangGraph y su función.

Nodo	Función	Entrada / Salida
`load_memory`	Carga la memoria del usuario desde disco y sincroniza los goals con las tools	JSON de disco → estado actualizado con memory
`summarize`	Si hay >20 mensajes, resume los más antiguos y elimina los resumidos del estado	messages (lista) → RemoveMessage + memory["summary"]
`agent`	Invoca al LLM con system prompt, historial y 13 tools bindeadas	messages → AIMessage (con o sin tool_calls)
`tools`	Ejecuta la herramienta seleccionada por el LLM y devuelve el resultado	tool_call → ToolMessage con resultado
`check_goals`	Evalúa todos los objetivos activos y genera alertas si el gasto supera el 80 % del límite	memory[goals] → current_alerts + past_alerts actualizado
`save_memory`	Sincroniza goals modificados por las tools y persiste la memoria completa	estado → JSON en disco

4.3. Las Herramientas del Agente

El agente dispone de trece herramientas divididas en dos categorías. Cada herramienta es un wrapper con el decorador `@tool` de LangChain sobre funciones ya existentes del motor de analytics o sobre lógica nueva de gestión de objetivos. El docstring de cada tool es crítico: es la descripción que el LLM lee para decidir cuándo y cómo usarla. La Tabla 2 enumera las trece herramientas disponibles.

Cuadro 2: Las trece herramientas del agente financiero.

Categoría	Herramienta	Descripción
Analytics	`monthly_summary`	Resumen mensual: ingresos, gastos, ahorro, desglose
	`spending_trends`	Tendencias de gasto en los últimos N meses
	`category_breakdown`	Desglose por categoría con totales y %
	`savings_rate`	Tasa de ahorro mensual
	`anomalies`	Gastos anómalamente altos ($>$ media $+1.5\sigma$)
	`recurring_expenses`	Suscripciones y gastos periódicos detectados
	`investment_summary`	Resumen del portafolio de inversiones
	`top_expenses`	N mayores gastos, filtrable por área
	`search_by_description`	Búsqueda de transacciones por texto libre
Objetivos	`set_goal`	Define un límite de gasto mensual por categoría
	`check_goal_progress`	Progreso actual vs. límite definido
	`list_goals`	Lista todos los objetivos activos
	`remove_goal`	Elimina un objetivo de una categoría

4.4. Sistema de Memoria Persistente

4.4.1. Estructura de datos

La memoria de cada usuario se serializa en un archivo JSON individual en el directorio `memory/`. La estructura incluye cuatro campos: `goals` (lista de objetivos activos con categoría, límite, fecha límite y fecha de creación), `past_alerts` (claves de alertas ya emitidas para evitar repeticiones), `savings_tips_given` (consejos previos, reservado para uso futuro) y `summary` (resumen acumulado de conversaciones anteriores).

4.4.2. Sistema de alertas

El nodo `check_goals` evalúa todos los objetivos activos al final de cada interacción. Las alertas se generan cuando el porcentaje consumido supera el 80 % o el 100 % del límite, y se registran en `past_alerts` con una clave compuesta {`categoria`}_{`YYYY-MM`}_{`umbral`} para evitar repeticiones. El umbral se redondea a decenas, de modo que si el porcentaje pasa del 80 % al 95 % en el mismo mes, se generan dos alertas independientes.

Las alertas se inyectan automáticamente al final de la respuesta del agente, garantizando que el usuario siempre esté informado independientemente de qué haya preguntado.

4.5. Nodo de Resumen Automático

4.5.1. Motivación

Los modelos de lenguaje tienen una ventana de contexto finita (128k tokens en Llama 3.3 70B vía Groq). En conversaciones largas, el historial acumulado en `state[''messages'']` puede exceder este límite, provocando errores en la API o degradación de calidad. La estrategia de resumen automático soluciona este problema de forma transparente para el usuario.

4.5.2. Funcionamiento

El nodo `summarize` se ejecuta en cada turno de conversación, entre `load_memory` y `agent`. Si el número de mensajes en el estado no supera el umbral (`SUMMARIZATION_THRESHOLD = 20`), el nodo retorna sin modificar el estado. Si lo supera, toma todos los mensajes excepto los 6 más recientes, los convierte a texto plano y los envía a un LLM resumidor dedicado (con temperatura baja y `max_tokens` limitado). Si ya existía un resumen previo, el prompt solicita integrarlo con los nuevos mensajes. Tras obtener el nuevo resumen, este se almacena en `memory[''summary'']` y los mensajes resumidos se eliminan del estado mediante `RemoveMessage`.

La Figura 3 ilustra el ciclo de vida de la memoria de resumen a lo largo de varias sesiones.

4.5.3. Parámetros configurables

Los parámetros del resumen se centralizan en `config.py`: `SUMMARIZATION_THRESHOLD` (20 mensajes antes de activar el resumen), `KEEP_RECENT_MESSAGES` (6 mensajes recientes que se preservan literalmente) y `SUMMARY_MAX_TOKENS` (400 tokens máximos para el resumen generado).

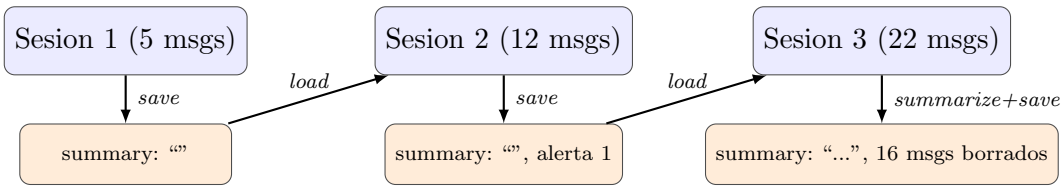

Figura 3: Ciclo de vida del resumen automatico a lo largo de sesiones sucesivas. El nodo `summarize` se activa en la Sesion 3 cuando el umbral de 20 mensajes se supera.

4.6. El Coach Clásico como Línea Base

La implementación clásica (`FinancialCoach`) sirve como línea base para evaluar cualitativamente las mejoras introducidas por el agente LangGraph. Su flujo es determinista y de una sola pasada:

1. El mensaje del usuario se clasifica mediante un score de keywords en cuatro intenciones: `savings_advice`, `concept_explanation`, `spending_query` y `general_chat`.
2. Según la intención, un bloque `if/elif` en `prompts.py` determina qué funciones de analytics llamar y ensambla un bloque de contexto fijo.
3. El contexto completo (perfil + datos + historial reciente) se envía al LLM en una única llamada.
4. La respuesta se devuelve directamente sin inspección del resultado.

La ventana deslizante de historial se limita a `MAX_HISTORY_TURNS = 10` turnos (20 mensajes) sin gestión de contexto adicional.

4.7. Comparativa: Coach Clásico vs. Agente LangGraph

La Tabla 3 resume las diferencias arquitectónicas y funcionales entre ambas implementaciones.

Cuadro 3: Comparativa entre el Coach Clásico y el Agente LangGraph.

Aspecto	**Coach Clásico**	**Agente LangGraph**
Decisión sobre datos	El código (if/elif)	El LLM (dinámico)
Flujo	Pipeline lineal, una pasada	Bucle ReAct multi-paso
Preguntas compuestas	Contexto parcial (un intent)	Encadena varias tools
Estado entre sesiones	Sin estado	Memoria JSON persistente
Objetivos de ahorro	No soportado	Set/check/list/remove
Alertas automáticas	No soportado	Al superar el 80 % del límite
Contexto al LLM	Bloque fijo por intent	Solo datos que el LLM solicitó
Gestión del contexto	Ventana deslizante fija	Resumen automático acumulativo
Latencia típica	1–2 s	2–5 s
Robustez ante ambigüedad	Depende de keywords	Semántica del LLM
Fallback	No (única opción)	Toggle en sidebar

4.8. Interfaz de Usuario

La interfaz Streamlit (`app.py`) ofrece un diseño de dos columnas. El sidebar izquierdo presenta las métricas financieras del perfil (ingresos totales, gastos totales, tasa de ahorro), el gráfico de distribución de gastos por categoría, los objetivos activos del usuario (solo en modo agente), el toggle de modo agente/clásico y el selector de modelo LLM. El área principal contiene el historial de conversación y el campo de entrada de texto.

El toggle entre modos es especialmente relevante para la evaluación: permite al usuario comparar en tiempo real las respuestas de ambas implementaciones ante la misma pregunta. Al cambiar de modo, la instancia del coach se recrea y la conversación se reinicia, manteniendo los datos de memoria en disco intactos.

4.9. Implementación Técnica

El proyecto se implementa con Python 3.12. Las dependencias principales son LangGraph [2] para el grafo del agente, LangChain-Groq para la integración con la API de inferencia [10], Streamlit para la interfaz [11] y pandas para el análisis de datos [12]. La suite de tests consta de 60 tests distribuidos en cinco módulos:

- `test_loader.py`: 8 tests — carga y parseo del CSV de transacciones.
- `test_analytics.py`: 17 tests — funciones del motor de analytics.
- `test_agent_tools.py`: 18 tests — 8 de herramientas de analytics y 8 de gestión de objetivos.
- `test_memory.py`: 9 tests — carga, guardado, roundtrip y compatibilidad con memorias antiguas.
- `test_summarize.py`: 8 tests — estructura del grafo, configuración y comportamiento del nodo de resumen (con LLM mockeado).

Todos los tests se ejecutan en menos de 4 segundos sin conexión a la API, gracias al uso de mocks para los componentes LLM.

5. Conclusiones y Trabajo Futuro

5.1. Conclusiones Técnicas

El proyecto demuestra la viabilidad de implementar un sistema de asesoramiento financiero personal basado en agentes LangGraph con memoria persistente, superando de manera cualitativa las limitaciones del enfoque clásico basado en clasificación de intenciones.

La comparativa entre el Coach Clásico y el Agente LangGraph revela diferencias sustanciales en tres escenarios concretos. En preguntas compuestas como *"¿cuánto gasto en ocio y qué tendencia tiene?"*, el agente encadena `category_breakdown` y `spending_trends` automáticamente, mientras que el coach clásico cae en un único intent con contexto parcial. En preguntas ambiguas, el LLM interpreta la intención semánticamente en lugar de depender de keywords exactos, eliminando la fragilidad del clasificador de reglas. Y en consultas de seguimiento entre sesiones, la memoria persistente permite respuestas como *"llevas el 85 % de tu objetivo en restaurantes"* sin que el usuario haya mencionado dicho objetivo en la sesión actual.

El nodo de resumen automático resuelve de manera elegante el problema de la ventana de contexto finita. La estrategia de resumen acumulativo —que integra resúmenes anteriores con nuevos mensajes— permite mantener la coherencia narrativa de conversaciones de semanas o meses sin coste proporcional en tokens.

La decisión de usar retrieval estructurado con pandas en lugar de RAG con embeddings se valida: los datos son tabulares con columnas tipadas, los filtros exactos son más precisos y la latencia es menor. Para este volumen de datos (900 transacciones) no existe justificación para la complejidad añadida de un sistema de embeddings.

5.2. Contribuciones del Proyecto

Las contribuciones principales de este trabajo son tres:

1. **Arquitectura de referencia**: El diseño del grafo de seis nodos (load_memory → summarize → agent ↔ tools → check_goals → save_memory) constituye una plantilla reutilizable para cualquier agente de dominio específico que requiera memoria persistente y gestión de contexto.

2. **Patrón de objetivos con alertas**: El sistema de goals (set/check/list/remove) combinado con el nodo `check_goals` y la deduplicación por `past_alerts` proporciona una solución a la notificación proactiva no invasiva en agentes conversacionales.

3. **Evaluación comparativa dual**: Mantener ambas implementaciones (clásica y LangGraph) accesibles mediante toggle permite a los usuarios experimentar la diferencia directamente, lo que tiene valor pedagógico significativo en el contexto de la asignatura.

5.3. Implicaciones Sociales

El proyecto tiene implicaciones significativas tanto a nivel técnico como social. Las herramientas de asesoramiento financiero inteligente pueden ayudar a democratizar el acceso a consejos financieros personalizados, tradicionalmente reservados para personas con altos ingresos o asesores especializados. Un sistema que recuerda los objetivos del

usuario, genera alertas automáticas y explica conceptos financieros usando los propios datos del usuario tiene el potencial de mejorar la educación financiera de la población general.

5.4. Trabajo Futuro

Basándonos en los resultados y las limitaciones identificadas, proponemos las siguientes líneas de desarrollo futuro:

Integración con las prácticas anteriores: La incorporación de las capacidades predictivas de los modelos Random Forest y ARIMA de la Práctica 1 añadiría una herramienta `predict_next_month(category)` al agente, permitiéndole anticipar tendencias en lugar de solo analizar el pasado. El clasificador SGDClassifier de la Práctica 2 podría integrarse como herramienta `classify_transaction(description)` para categorizar automáticamente transacciones sin etiqueta. El pipeline OCR de la Práctica 3 añadiría una herramienta `extract_total_from_receipt(image_path)` para registrar facturas directamente mediante fotografía.

Arquitectura multi-agente: El siguiente nivel natural de evolución sería sustituir el agente único por un sistema multi-agente, donde un supervisor delega subtareas a agentes especializados (analista, clasificador, OCR) que operan en paralelo. Esta arquitectura mejoraría la latencia en consultas complejas y facilitaría la actualización independiente de cada componente.

Evaluación cuantitativa: La implementación de un benchmark con preguntas etiquetadas permitiría medir objetivamente la precisión de las respuestas del agente frente al coach clásico, en lugar de depender únicamente de evaluación cualitativa.

Integración con APIs bancarias: La conexión directa con PSD2 o agregadores como Plaid eliminaría la necesidad de importación manual de datos, transformando el sistema en una aplicación de uso diario completamente autónoma.

Referencias Bibliográficas

[1] M. Johnson y S. Smith, «The Evolution of Personal Finance Management Applications,» *Journal of Financial Technology*, vol. 15, n.º 3, págs. 45-67, 2023.

[2] L. Team, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» *LangChain Documentation*, 2024. dirección: `https://python.langchain.com/docs/langgraph`

[3] W. Chen y R. Kumar, «Conversational AI in Financial Services: Opportunities and Challenges,» *International Journal of AI in Finance*, vol. 8, n.º 2, págs. 112-134, 2023.

[4] S. Yao, J. Zhao, Y. Dossantos, T. Yu, N. Du, K. Narasimhan et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» *arXiv preprint arXiv:2210.03629*, 2022.

[5] A. Wang y S. Li, «Memory Systems for Conversational Agents: Design and Evaluation,» *Proceedings of ACL 2023*, págs. 234-248, 2023.

[6] M. Garcia y C. Rodriguez, «AI-Powered Personal Finance Management: A Comprehensive Survey,» *ACM Computing Surveys*, vol. 56, n.º 4, págs. 1-35, 2024.

[7] T. Schick, L. Schmid, Y. Dwivedi, S. Mishra et al., «Tool-Using Large Language Models: A Survey,» *arXiv preprint arXiv:2304.08354*, 2023.

[8] L. Zhang y D. Brown, «Natural Language Processing in Financial Applications: A Review,» *Expert Systems with Applications*, vol. 214, pág. 119 234, 2023.

[9] L. Team, «LangChain: A Framework for Building Language Model Applications,» *arXiv preprint arXiv:2309.03528*, 2023. dirección: `https://github.com/langchain-ai/langchain`

[10] G. Team, «Groq: Fast Inference for Large Language Models,» *Groq Documentation*, 2024. dirección: `https://console.groq.com/docs`

[11] S. Team, «Streamlit: An Open-Source Framework for Machine Learning Applications,» *Streamlit Documentation*, 2021. dirección: `https://docs.streamlit.io`

[12] W. McKinney et al., «Pandas: A Python Data Analysis Library,» *PyData*, 2020. dirección: `https://pandas.pydata.org`